KAPITEL 2: WIE FUNKTIONIERT DAS WORLD WIDE WEB?

LERNZIELE

- Die Begriffe WWW und Hypertext-System erklären können.
- Das HTTP-Protokoll und dessen Bestandteile erläutern können.
- Wichtige Software-Bausteine des WWW nennen und deren Aufgabe erläutern können.
- Aufbau von URLs und URIs darstellen und in Beispielen anwenden können.



2.1 BEGRIFFE

Was genau ist das Word Wide Web?

Definition World Wide Web (WWW)

- Das World Wide Web ist ein Interaktives Informationssystem, das den weltweiten Austausch digitaler Dokumente ermöglicht.
- Es besteht aus sog. Hypertext-Systemen. Im WWW wird ein Hypertext-System Website genannt. Eine Website umfasst i.d.R. mehrere zusammenhängende Webdokumente.
- Die Homepage (Startseite) ist dabei der zentrale Einstiegspunkt einer Website, das WWW insgesamt ein gigantisches und in seinen Verzweigungen für den Nutzer nicht nachvollziehbares Hypertext-System.
- Alle Webdokumente befinden sich auf speziellen an das Internet angebundenen Rechnern, von denen sie mithilfe von Web-Server-Diensten bereitgestellt werden.

Definition WWW (Fortsetzung)

- Um auf die Dokumente des WWW zuzugreifen, ist ein Browser nötig.
- Die Adressierung eines Webdokuments, d.h. eines beliebigen Objektes im WWW, erfolgt durch die URL (Uniform Resource Locator).
- Für die Hypertext-Dokumente im WWW wurde eine einfache und plattformunabhängige Sprache entwickelt: HTML (Hypertext Markup Language).
- Um die Dokumente bzw. Objekte schnell via Hyperlinks über das Netz aufzurufen, wurde für die Übertragung der Daten das Hypertext-Transferprotokoll (HTTP) entwickelt.

Die Anfänge des WWW in Auszügen:

Jahr	Ereignis
1945	Erstes Hypertextsystem MEMEX von Vennevar Bush
1968	Doug Engelbart entwickelt NLS, der erste Hypertext-basierte Prototyp; er erfindet hierfür die Computer-Maus
1989	Erstes Memo zu einem Hypertext-Dokumentenverwaltungssystem am Kernforschungszentrum CERN von Tim Berners-Lee
1990	**Erster WWW-Server/-Browser durch Tim Berners-Lee und Robert Cailliau – Geburt des World Wide Web**
1993	NCSA Mosaic, erster WWW-Browser mit GUI erscheint
1994	Gründung des World Wide Web Consortiums (W3C)

Hypertext-Dokumente

Herkömmliche Textdokumente werden typischerweise sequentiell gelesen oder verändert. Sie liegen in der Regel lokal, d.h. auf dem eigenen Rechner, vor.

Hypertext-Dokumente sind im Gegensatz dazu Text-Dokumente, die Querverweise (sog. Hyperlinks) auf andere Hypertext-Dokumente enthalten.

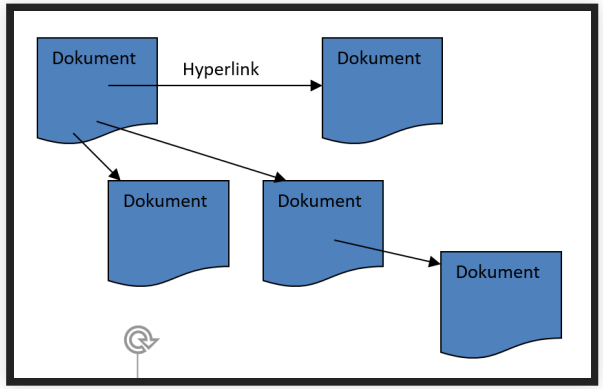
Ein Hypertext-Dokument ist in der Regel "multimedial", d.h. es kann neben Text auch Grafiken, Bilder, Audio-Clips, Video-Clips und Animationen enthalten.

Hypertext-Dokumente werden mit einer eigenen Sprache beschrieben, der Hypertext Markup Language (HTML). Diese wird in Kapitel 3 behandelt.

Hypertext-System

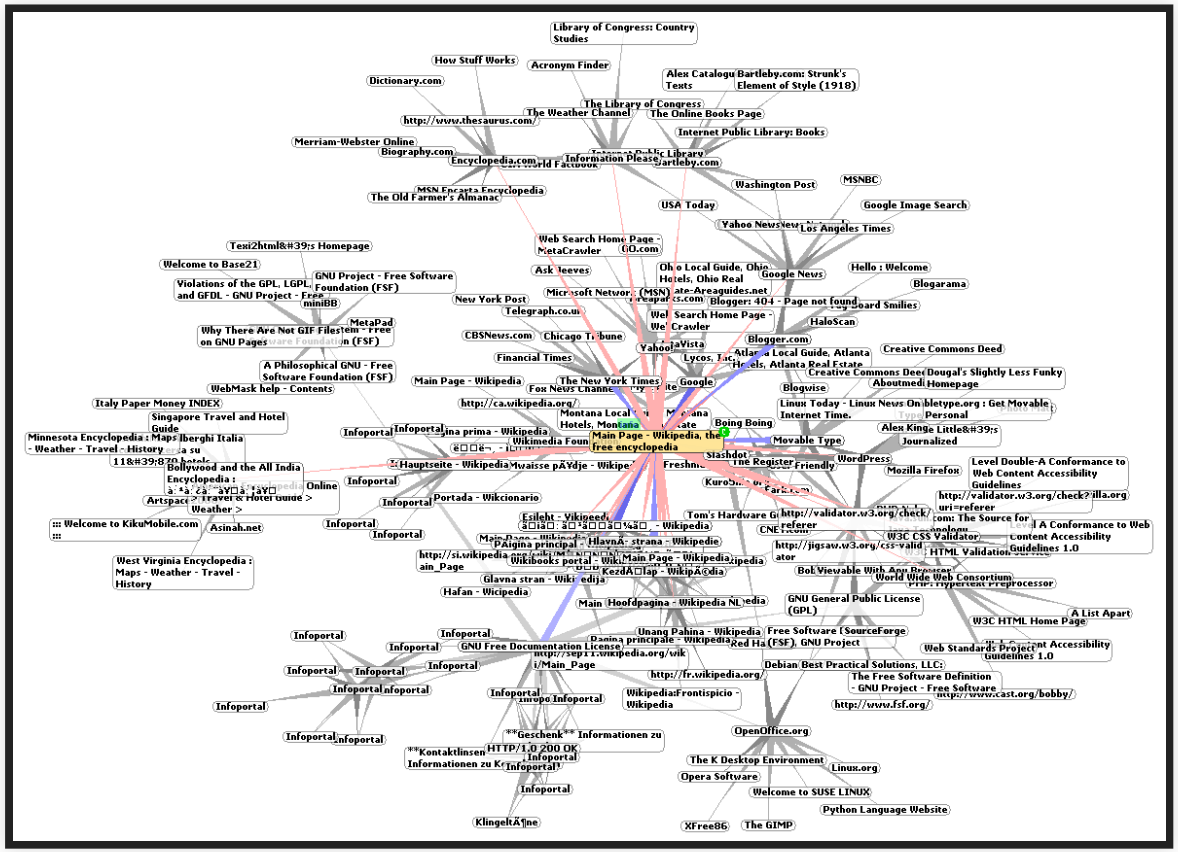
Ein Hypertext-System ist ein Software-System, welches den Zugriff auf Hypertext-Dokumente ermöglicht. Entsprechend ist das WWW ein (weltweit) verteiltes Hypertext-System, auch Hypermedia-System genannt.

Die Verteilung ergibt sich durch die Speicherung der Dokumente auf beliebigen Servern im Internet. Das Hypertext-Dokument funktioniert trotz der Verteilung, da Hyperlinks mächtig genug sind, um jeden Hypertext-Server im Internet anzusprechen. Durch die Verteilung ergeben sich ggf. Probleme, da z.B. ein Server nicht erreichbar sein kann oder ein Dokument auf einem Server gelöscht oder umbenannt sein kann.



Beispiel: Wikipedia

Wikipedia (z.B. http://www.wikipedia.de) ist eines der größten Hypertext-Systeme. Seine Serverstruktur ist nachfolgend abgebildet (Quelle hier):





2.2 HTTP(S)

HTTP steht für Hypertext Transfer Protocol. Es wurde dem Namen entsprechend speziell für die Übertragung von Hypertexten entwickelt.

HTTP basiert auf TCP/IP und hat entsprechend Ports als Kommunikationsendpunkte. Die für HTTP-Server weltweit unterstützten Standard-Ports sind *Port 80* für HTTP und *Port 443* für HTTPS. HTTPS-Nachrichten sind zusätzlich verschlüsselt (S steht für „secure“).

HTTP-Nachrichten sind Text-Nachrichten, d.h. sie sind für Menschen gut lesbar. Jede HTTP-Nachricht hat einen Nachrichtenkopf (*header*) und einen Nachrichtenkörper (*body*) mit den Nutzdaten.



HTTP-Befehle

HTTP unterstützt eine Reihe von **Befehlen** (sogenannte HTTP-Methoden), von denen vier im Kapitel zum Thema REST genutzt werden. Einer der Befehle, der Befehlt `GET`, wird im Folgenden anhand eines Beispiels erläutert. Mit einem modernen Browser wie beispielsweise Chrome oder Firefox (Entwicklertools nutzen) kann das nachfolgende Beispiel leicht nachvollzogen werden.



Laden der Startseite von Wikipedia: GET https://www.wikipedia.de

GET sendet eine HTTP-Nachricht (Request) an den Server. Der Server antwortet mit einer Nachricht (Reply) mit einem *header* und der HTML-Seite im *body*.

Request Header (Ausschnitt):

```
GET / HTTP/1.1
Host: www.wikipedia.de
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:68.0) Gecko/2010010
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Cookie: _pk_id.3.9878=...
```

Reply Header (Ausschnitt):

```
HTTP/1.1 200 OK
Server: nginx/1.6.2
Date: Sat, 20 Jun 2020 19:23:05 GMT
Content-Type: text/html; charset=UTF-8
```

Reply body (Ausschnitt):

```
<!DOCTYPE html>
...
<body>
...
```

Standardmethoden

Die häufigst anzutreffenden HTTP-Methoden sind GET und POST. Weitere wichtige sind PUT, DELETE und OPTIONS:

Methode	Interpretation
GET	Anfrage der entsprechenden Ressource
POST	Anfrage mit Anfrage-Datenpaket (dieses kann z.B. Formulardaten enthalten).
PUT	Anfrage mit Anfrage-Datenpaket, soll die entsprechende Ressource durch das Paket ersetzen.
DELETE	Anfrage der entsprechenden Ressource
OPTIONS	Die Methode wird verwendet, um die Kommunikationsoptionen für die Zielressource zu erfragen.

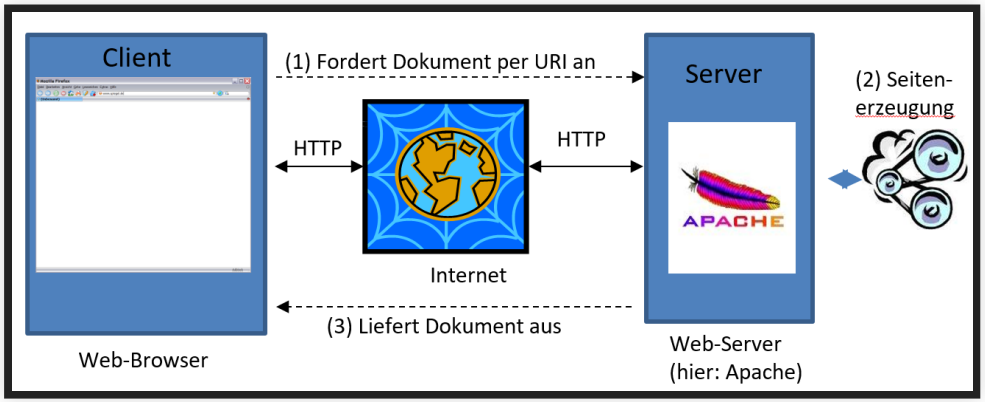
Standardantworten

Neben der bekannten Standardantwort "404 - Not Found" gibt es eine vielzahl möglicher Antworten auf eine HTTP-basierte Anfrage. Eine umfassende Liste findet sich auf Wikipedia, eine Auswahl findet sich im folgenden:

Code	Interpretation
200	Erfolgreiche Anfragebearbeitung mit Ergebnis
204	Erfolgreiche Anfragebearbeitung ohne Ergebnis
301	Die angefragte Ressource steht ab sofort unter einer neuen Adresse (angegeben über Location-Header) zur Verfügung
400	Fehlerhafte Anfrage
403	Anfrage fehlt es an Berechtigung
404	Die Ressource konnte nicht gefunden werden
500	Interner Server Fehler
501	Funktionalität nicht implementiert

2.3 SOFTWARE-BAUSTEINE DES WWW

Prinzipielle WWW-Interaktion



Prinzipiell funktioniert das WWW folgendermaßen:

1. Ein Webbrowser fordert über einen sogenannten URI (unique resource identifier) ein Hypertext-Dokument vom Webserver an.
2. Der Server (im Beispiel Apache) prüft, ob das Dokument unverändert geliefert werden kann ("statische" Webseite), oder ob das bezeichnete Dokument ein Generator-Skript ist, welches seinerseits ein HTML-Dokument erzeugt.
3. Das erzeugte Dokument wird an den Web-Browser zurückgeschickt und von diesem angezeigt.



Populäre Webbrowser sind aktuell: Chrome, Safari, Edge, Firefox.

Auf W3Schools sieht man, wie populär welcher Browser ist. Man erkennt, dass aktuell Chrome die Nase vorne hat, was neben der Qualität des Browsers sicher auch an der Dominanz von Android auf dem Smartphone-Markt liegt.

Populäre Webserver sind Apache, nginx (sprich: Engine X) und der IIS von Microsoft (siehe NETCRAFT).

IDENTIFIKATION VON WEB-RESSOURCEN

URIs (*unique resource identifier*) ermöglichen die weltweit eindeutige Identifikation einer Ressource. Eine Ressource ist hier eine Webseite, kann aber auch eine Datei oder eine Datenbankabfrage sein.

Beispiel URI: `mailto:someone@example.com`

Neben URIs gibt es URLs (*unique resource locator*) als Teilmenge der URIs. Ein URL enthält den Weg zum Auffinden einer Ressource. Wir betrachten im Weiteren ausschliesslich URLs, auch wenn der oben genannte URI z.B. mit einem Anchor Tag genutzt werden kann, um eine E-Mail aus einer Webseite heraus abzuschicken.

Einen guten Vergleich von URIs, URLs und URNs (Ressource Namen) findet man z.B. hier.

Aufbau eines URL:

Die allgemeine Syntax ist:

```
[Protokoll://]computer-name[:port][/dokumenten-pfad][Query][Fragment]
```

Beispiel:

Das HTTP-Protokoll soll genutzt werden, um auf die Datei "index.html" im Verzeichnis "test" auf dem Server localhost mit Port "8080" zuzugreifen.

```
http://localhost:8080/test/index.html
```

Eine **Query** enthält Paare aus Schlüsseln (keys) und Werten.

Beispiel: http://localhost:8080/test/index.html**?name=Maier&vorname=Erna**

Die Query übergibt zum Schlüssel "name" den Wert "Maier" und zum Schlüssel "vorname" den Wert "Erna" an den Server.

Ein **Fragment** dient dazu, eine bestimmte Position innerhalb eines Hypertext-Dokuments zu bezeichnen, typischerweise mit dem Ziel, die Position im Browser anzuzeigen. Dies macht insbesondere bei großen HTML-Dokumenten Sinn. Im Beispiel wird die Position "abschnitt3" in der Datei index.html bezeichnet. Die Position muss als HTML-Element mit der id "abschnitt3" existieren.

Beispiel für ein Fragment:

```
http://localhost:8080/test/index.html#abschnitt3
```

Hinweis: Fragment-Information wird nicht zum Server übertragen!

ENTWICKLUNG DES WWW UND DER BROWSER-TECHNOLOGIEN

Neben der Verbindung zu Webservern (zumeist durch Nutzung des HTTP-Protokolls) ist die wesentliche Aufgabe eines Webbrowsers die Darstellung von Webseiten mit heutzutage multimedialen Inhalten sowie die Entgegennahme von Eingaben über Tastatur, Touchscreen bzw. Maus. Entsprechend enthält ein Webbrowser standardmäßig Interpreter für HTML, CSS und JavaScript.

2.4 NAMENSAUFLÖSUNG ÜBER DAS DOMAIN NAME SYSTEM (DNS)



Ein Webbrowser-Benutzer gibt beim Surfen einen URI (z.B. http://www.spiegel.de) ein, der einen Hostnamen enthält.

Wie wird nun beispielsweise der hinter "www.spiegel.de" stehende Webserver gefunden?

Schlussendlich wird eine IP-Adresse benötigt, um über IP bzw. TCP oder HTTP Nachrichten an den Server senden zu können.

ping ist ein Shell-Utility, mit welchem man testen kann, ob ein Server antwortet:

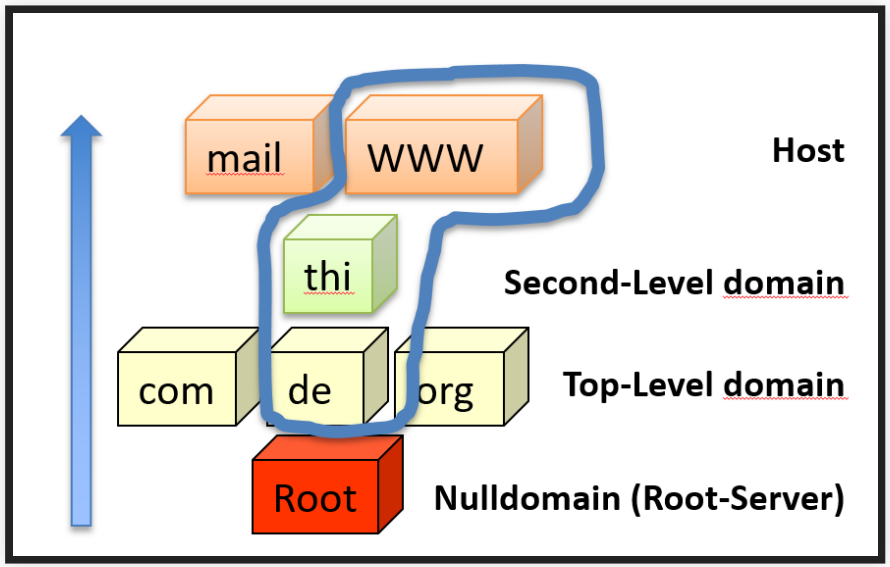
```
ping www.spiegel.de

Ping wird ausgeführt für aacfb9d106f4.link11.de [128.65.210.185] mit 32 Byte
Antwort von 128.65.210.185: Bytes=32 Zeit=17ms TTL=56

Ping-Statistik für 128.65.210.185:
    Pakete: Gesendet = 4, Empfangen = 4, Verloren = 0
    (0% Verlust),
Ca. Zeitangaben in Millisek.:
    Minimum = 16ms, Maximum = 23ms, Mittelwert = 18ms
```

Um die Umsetzung von Server-Namen in IP-Adressen zu ermöglichen, wurde ein weltweiter hierarchischer Namensraum sowie ein entsprechender Dienst – das **Domain Name System (DNS)** – entwickelt.

Die folgende Abbildung verdeutlicht die Struktur dieses Namensraums:



Die sog. „Nulldomain“ stellt den Einstieg dar, das sind Namens-Server, die auch als Root-Server bezeichnet werden (siehe http://www.root-servers.org/). Von diesen ausgehend wird der Name rückwärts interpretiert, wie im Bild dargestellt. Man erkennt, dass der www-Host nur ein Server unter mehreren sein kann. Im Bild ist noch der mail-Server zu sehen.

Wer löst den Namen auf?

Bei der Namensauflösung wird ein im Betriebssystem konfigurierter Namensdienst-Server angefragt. Dieser wiederum kommuniziert wie oben beschrieben mit den Domain-Servern. Mehr Information findet man z.B. unter http://www.netplanet.org/adressierung/dns.shtml.



2.5 ÜBUNGEN

ÜBUNG: WWW

Wofür steht WWW?

- [ ] WWW steht für WeltWeites Weitverkehrsnetz
- [ ] WWW steht für WWW-Netzwerk
- [ ] WWW steht für World Wide Web und bezeichnet ein auf dem Internet basierendes Client-Server Informationssystem

ÜBUNG: WEBARCHITEKTUR

Welche Komponenten müssen in einer Webarchitektur vorhanden sein?

- [ ] ApplicationServer
- [ ] Firewall
- [ ] Webserver
- [ ] Proxy
- [ ] Webbrowser
- [ ] Gateway

ÜBUNG: HYPERTEXT

Was ist ein Hypertext? Wählen Sie die am besten passende Antwort!

- [ ] ein Hypertext ist ein Dokument mit Text und Multimedia-Inhalten, welches zudem Verweise auf andere Hypertexte enthalten kann.
- [ ] ein Hypertext ist ein Textdokument mit Links
- [ ] ein Hypertext ist ein baumartig organisierter Text

ÜBUNG: HTTP-METHODEN

Welches sind gültige HTTP-Methoden? Bitte die Antwort wählen, die am besten zur Frage passt!

- [ ] SEND und RECEIVE
- [ ] GET und PUT
- [ ] GET und POST

ÜBUNG: SOFTWARE-BAUSTEINE DES WWW

Was sind Software-Bausteine des WWW? Bitte die Antwort wählen, die am besten zur Frage passt!

- [ ] HTTP-Client und -Server
- [ ] URL und URI
- [ ] Webbrowser und Webserver

PROTOKOLLE

Im folgenden Download liegt ein Beispiel bereit, mit dem verschiedene Aspekte der vorgestellten Technologien analysiert werden können. Hierfür ist als Ausführungsumgebung Docker-Desktop notwendig. Nutzen Sie die Anweisungen in der README.

Nutzen Sie das bereitgestellte Projekt-Archiv http-call.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Ergebnis des Aufrufs `dig de NS`

```
; <<>> DiG 9.14.12 <<>> de NS
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 58081
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;de.                    IN    NS

;; ANSWER SECTION:
de.             4502    IN    NS    z.nic.de.
de.             4502    IN    NS    f.nic.de.
de.             4502    IN    NS    n.de.net.
de.             4502    IN    NS    a.nic.de.
de.             4502    IN    NS    l.de.net.

;; Query time: 71 msec
```

Namenserver für Anfragen bzgl. DE-Domains

Ergebnis des Aufrufs `dig thi.de NS`

```
; <<>> DiG 9.14.12 <<>> thi.de NS
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 60277
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;thi.de.                IN    NS

;; ANSWER SECTION:
thi.de.             4502    IN    NS    dns1.lrz.de.
thi.de.             4502    IN    NS    dns2.lrz.bayern.
thi.de.             4502    IN    NS    dns3.lrz.eu.

;; Query time: 75 msec
```

Namenserver für Anfragen bzgl. Subdomains unter thi.de

Ergebnis des Aufrufs `dig thi.de A`

```
; <<>> DiG 9.14.12 <<>> thi.de A
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 34296
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;thi.de.                IN    A

;; ANSWER SECTION:
thi.de.             4502    IN    A    194.94.240.181

;; Query time: 34 msec
```

IP-Adresse des Servers thi.de

Ergebnis des Aufrufs `dig online-lectures-cs.thi.de A`

```
; <<>> DiG 9.14.12 <<>> online-lectures-cs.thi.de A
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 10026
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;online-lectures-cs.thi.de.    IN    A

;; ANSWER SECTION:
online-lectures-cs.thi.de. 4502    IN    A    194.94.240.192

;; Query time: 2 msec
```

IP-Adresse des Servers online-lectures-cs.thi.de

Ergebnis von TCP-Dump nach cURL

```
11:52:15.446136 IP a232a6667592.59696 > http-call_nginx_1.http-
call_default.80: Flags [P.], seq 2429739823:2429739900, ack 576998787,
win 502, options [nop,nop,TS val 806929392 ecr 3251102293], length 77:
HTTP: GET /test.txt HTTP/1.1
    0x0000:  0242 ac1a 0003 0242 ac1a 0002 0800 4500  .B.....B......E.
    0x0010:  0081 afcc 4000 4006 3271 ac1a 0002 ac1a  ....@.@.2q......
    0x0020:  0003 e930 0050 90d2 e32f 2264 4d83 8018  ...0.P.../"dM...
    0x0030:  01f6 58ad 0000 0101 080a 3018 c3f0 c1c7  ..X.......0.....
    0x0040:  e255 4745 5420 2f74 6573 742e 7478 7420  .UGET./test.txt.
    0x0050:  4854 5450 2f31 2e31 0d0a 486f 7374 3a20  HTTP/1.1..Host:.
    0x0060:  6e67 696e 780d 0a55 7365 722d 4167 656e  nginx..User-Agen
    0x0070:  743a 2063 7572 6c2f 372e 3637 2e30 0d0a  t:.curl/7.67.0..
    0x0080:  4163 6365 7074 3a20 2a2f 2a0d 0a0d 0a    Accept:.*/*....
...
```

```
...
11:52:15.450002 IP http-call_nginx_1.http-call_default.80 > a232a6667592.
59696: Flags [P.], seq 1:243, ack 77, win 509, options [nop,nop,TS val
3251102297 ecr 806929392], length 242: HTTP: HTTP/1.1 200 OK
    0x0000:  0242 ac1a 0002 0242 ac1a 0003 0800 4500  .B.....B......E.
    0x0010:  0126 db1a 4000 4006 067e ac1a 0003 ac1a  .&..@.@..~......
    0x0020:  0002 0050 e930 2264 4d83 90d2 e37c 8018  ...P.0"dM....|..
    0x0030:  01fd 5952 0000 0101 080a c1c7 e259 3018  ..YR.........Y0.
    0x0040:  c3f0 4854 5450 2f31 2e31 2032 3030 204f  ..HTTP/1.1.200.O
    0x0050:  4b0d 0a53 6572 7665 723a 206e 6769 6e78  K..Server:.nginx
    0x0060:  0d0a 4461 7465 3a20 5765 642c 2033 3020  ..Date:.Wed,.30.
    0x0070:  4465 6320 3230 3230 2031 313a 3532 3a31  Dec.2020.11:52:1
    0x0080:  3520 474d 540d 0a43 6f6e 7465 6e74 2d54  5.GMT..Content-T
    0x0090:  7970 653a 2074 6578 742f 706c 6169 6e0d  ype:.text/plain.
...
```

```
...
    0x00a0:  0a43 6f6e 7465 6e74 2d4c 656e 6774 683a  .Content-Length:
    0x00b0:  2031 330d 0a4c 6173 742d 4d6f 6469 6669  .13..Last-Modifi
    0x00c0:  6564 3a20 4d6f 6e2c 2030 3520 4f63 7420  ed:.Mon,.05.Oct.
    0x00d0:  3230 3230 2030 363a 3130 3a31 3620 474d  2020.06:10:16.GM
    0x00e0:  540d 0a43 6f6e 6e65 6374 696f 6e3a 206b  T..Connection:.k
    0x00f0:  6565 702d 616c 6976 650d 0a45 5461 673a  eep-alive..ETag:
    0x0100:  2022 3566 3761 6238 6338 2d64 220d 0a41  ."5f7ab8c8-d"..A
    0x0110:  6363 6570 742d 5261 6e67 6573 3a20 6279  ccept-Ranges:.by
    0x0120:  7465 730d 0a0d 0a48 656c 6c6f 2c20 576f  tes....Hello,.Wo
    0x0130:  726c 6421                                rld!
```

Zu sehen sind die Datenpakete die im rahmen der Anfrage über TCP zw. Client (cURL) und Server ausgetauscht werden. Diese enthalten die entsprechenden HTTP-Requests und HTTP-Responses für die Anfrage von `http://nginx/test.txt`.

```
0x0040:  e255 4745 5420 2f74 6573 742e 7478 7420  .UGET./test.txt.
0x0050:  4854 5450 2f31 2e31 0d0a 486f 7374 3a20  HTTP/1.1..Host:.
```

Neben teilen des TCP-Headers ist hier `GET /test.txt HTTP/1.1` erkennbar.

